Apple Pay JS wrapper

Overview

This is the API documentation for the client-side JS wrapper for Apple Pay. This library should simplify the handling of Apple Pay for merchants.

The Apple Pay JS wrapper works in conjunction with Windcave Restful API, as after creating a session the merchants must grab the link with **rel "ajaxSubmitApplePay"** and pass it into the wrapper's `create` function.

Preparations

The Apple Pay JS wrapper uses Apple Pay SDK JS to construct the Apple Pay button. Thus, to make sure everything works as expected the merchant needs to make sure they whitelist the following source in their Content Security Policy:

- `script-src` https://applepay.cdn-apple.com/jsapi/

If merchant doesn't use Content-Security-Policy at all, they won't need to do any CSP changes

Domain validation

Before the Apple Pay can be displayed the merchant's domain has to be validated.

Follow the steps mentioned here: https://www.windcave.com/developer-ecommerce-drop-in#apple_pay_domain_registration

After all the previous steps are done, We will provide the **Apple Pay for Web Internal Merchant Id**. The merchant will pass this value when they call the `create` function of Apple Pay JS wrapper.

Integration

To use the Apple Pay JS wrapper first of all it needs to be loaded by the page. To do that the merchant needs to add a script loading tag into the <head> element of their page.

```
<script src="https://sec.windcave.com/js/windcavepayments-applepay-
v1.js"></script>
```

They can also use "async" or "defer" attributes on that script to make sure the script loading process doesn't block the page loading.

Merchants have to reference them from our domains, they are **not allowed** to download those JS files and serve from their own servers.

As mentioned, the Apple Pay JS wrapper works in conjunction with Restful Session.

To be able to create an Apple Pay button using this wrapper, firstly the merchant has to create a new payment session using Restful Create Session request. This request has to be sent from the server (i.e. S2S request)

The response to that request looks like this:

Restful API Create Session response


```
{
    "id": "00001200002116585f8d891123456787",
    "state": "init",
    "links": [{
            "href":
"https://sec.windcave.com/api/v1/sessions/00001200002116585f8d891123456787"
,
            "rel": "self",
            "method": "GET"
        }, {
            "href":
"https://sec.windcave.com/pxmi3/F0AD730870909329421 5B271C6E087134C33EB01576
9FCFBA5322EE8F69378EB1D895FD231D9A10C116A6DA19F34F460A",
            "rel": "hpp",
            "method": "REDIRECT"
        }, {
            "href":
"https://sec.windcave.com/pxmi3/F5A2BEDE53131E04066A068518F8D02BC422773FE54
27CE19E05A5472028EABF01AC08C38D6378B26B6FD68134DCC0FBD",
            "rel": "seamless_hpp",
            "method": "REDIRECT"
        }, {
```

```
            "href":
"https://sec.windcave.com/pxmi3/F02CBA539AA694D6ECDEA85D44CCE23679B11B3ADEA
E7484F34E591DF44D96E0ADC35F47CFD7AC50F0C10805AE08D964C",
            "rel": "submitCard",
            "method": "FORM_POST"
        }, {
            "href":
"https://sec.windcave.com/pxmi3/F6AF0F75C5EE172E1E6C29B4AAF60023A48BFA2103B
78F50DF414F5D93902847D7DAF157683429F15E53995AC645D5DA1",
            "rel": "paymentMethod:paypal",
            "method": "REDIRECT"
        }, {
            "href":
"https://sec.windcave.com/mh/F105F622A16AA024CCCC4D7BCF4077B687C8E180886CF7
6C4DEB488635298070089E907543745D2BB7B60845E9C6BED34",
            "rel": "ajaxSubmitCard",
            "method": "AJAX_POST"
        }, {
            "href":
"https://sec.windcave.com/mh/FA0F72460E5A7D66289271BBD5E7F625EE767DC8A0FEE9
2E8E24496D8E0C20BC3E6601EE2D2CF7E9F3074F6515D131923",
            "rel": "ajaxSubmitGooglePay",
            "method": "AJAX_POST"
        }, {
            "href":
"https://sec.windcave.com/mh/FBA081BB5FC6805CD9DC88DA56A14020B01492B1C4ED27
3452DF9E668BD9334E07ED370DD3E7F10B007C0820A79DE415D",
            "rel": "ajaxSubmitApplePay",
            "method": "AJAX_POST"
        }
    ]
}
```


The merchant then has to loop through elements of `links` array and find the element that has "rel" field set to `ajaxSubmitApplePay` and pass it back to the front-end for constructing the button.

Once link is received and JS file is loaded, to create the actual Apple Pay wrapper component the merchant needs to call the method `WindcavePayments.ApplePay.create` . This method returns a Windcave Apple Pay controller that should be stored into a variable for future use.

The interface of `WindcavePayments.ApplePay.create` method is as follows:

```
  WindcavePayments.ApplePay.create(options, onSuccess, onFailure,
onPaymentStart, onValidateMerchant)
```

where:

- `options` is a JS object that contains all the necessary settings for the Apple Pay wrapper to operate (details below)
- `onSuccess` is a callback that is triggered when the Apple Pay transaction has finished. This DOES NOT mean that the transaction was authorized, just that it has successfully been finished - to check the actual outcome the merchant has to run a Query Session request from their back-end. This callback will get three arguments - `state` (the data submission state), `finalURL` (the final URL received from

Windcave after the data submission) and `outcomeNotificationFunction` . More information on the `onSuccess` callback below.

- `onError` is a callback that is triggered when the Apple Pay transaction has failed. This callback will get two arguments - `stage` the processing stage that the failure has occurred at, and `errorMessage` which contains a description of the failure. Possible values that `stage` argument can take are:
  - `setup` - the error has occurred during initialization of Apple Pay session
  - `pre-submit` – the error has occurred after the customer has approved the transaction but before it has been submitted to Windcave for processing
  - `submit` - the error has occurred after the data was submitted to Windcave. One of possible issues is Windcave server has triggered 3DSecure, but the JS wasn't able to process that response.
  - `transaction` - the error has occurred while processing the final response from the Windcave
  - `payment-start` – the payment has been cancelled during the processing of the Payment Start call

- `onPaymentStart` is a callback that is triggered when the customer has pressed Apple Pay button right before we display the Apple Pay payment sheet. This gives merchant chance for final preparations or to cancel the Apple Pay transaction before it has started. The callback has three arguments:
  - `paymentMethod` - the payment method that has been selected by the customer. For Apple Pay the passed value is always "applePay"
  - `next` - the function to keep processing the selected payment method. Once the merchant's code finished preparation, it needs to call next with no arguments to let the wrapper continue with the payment process.
  - `cancel` - the function to cancel processing. If for whatever reason the code decides that they don't want to proceed with the payment, the cancel has to be called with no arguments. In that case the merchant should expect `onError` to be triggered with `payment-start stage`.
- `onValidateMerchant` is the callback that is triggered when the Apple Pay transaction is started and it is about to start the merchant validation process. This is the latest point where the merchant can go and asynchronously create the session to get the AJAX POST url and update ApplePay options object with the correct URL in case their integration requires to create the session at the last moment. The callback has two arguments:
  - `next` - the function to keep processing the selected payment method. Once the merchant's code finished, it needs to call next with no arguments to let the wrapper continue with the payment process.
  - `cancel` - the function to cancel processing. If for whatever reason the code decides that they don't want to proceed with the payment, the cancel has to be called with no arguments. In that case the merchant should expect `onError` to be triggered with `pre-submit stage`.

Generally, getting `onError` with `stage` values `setup` , `pre-submit` and `payment-start` mean that the failure has occurred before the session has been finished, i.e. if the merchant got another link for the Create Session response for handling another payment

methods, then another payment method can still be attempted for the same session. All the other `stage` values mean the session has been finished and to retry the merchant has to create a new session.

If the integration requires creating the session and obtaining the AJAX POST link at the moment, it might be tempting to run an async request from `onPaymentStart`. However, that is not going to work - `onPaymentStart` has to be a linear, synchronous function which calls `next`, i.e. it is more suitable for some form fields validation more than anything else.

The reason for that is Apple Pay has to be started by user input. Attempt to delay the call of `next` until some asynchronous process finishes will delay the start of Apple Pay, hence breaking "start by user input" flow. This will lead to Apple Pay failing to start.

For the last minute obtaining of AJAX POST link the `onValidateMerchant` callback has to be used instead. It is triggered when the Apple Pay is already started but before the AJAX POST link is needed (i.e. before the data is being submitted to Windcave), and thus does not have that strict requirement, allowing for async functionality to be triggered and for delaying of `next` call.

The `onSuccess` callback plays a few important roles here.

1. It notifies merchant's code of the current status of the submission. The code should expect to receive this callback at most two times (as of 2024) . The first time (optional) is when 3DSecure is about to start (this will be the case only if the merchant's API user is configured to run 3DSecure for Apple Pay transactions) and `onSuccess` will get called with the first argument set to `3DSecure` . The second time is when the processing has been finished, in which case `onSuccess` will get called with the first argument set to `done` .
2. It is the way for JS wrapper to get confirmation from the merchant whether the transaction was authorized or not. JS wrapper needs this information to let Apple Pay know the outcome, so that the payment sheet could display correct result icon before being closed.

The 2nd role is very important, and it is important for merchant's code to provide the correct result. **Failure to provide the correct outcome or attempt to redirect to the result page without letting JS wrapper know will result in Apple Pay showing "Payment Failed" text and a decline icon, even though the actual financial transaction might have been successful.**

Apple Pay JS wrapper provides different options how `onSuccess` callback can be implemented and how the wrapper can be notified of the transaction result.

Option #1. As mentioned, `onSuccess` gets three arguments - `state` , `finalURL` and `outcomeNotificationFunction` . `outcomeNotificationFunction` is a function that gets a single Boolean argument, which indicates whether transaction was successful or not. Once final result is available, merchant's code can call this function which will immediately pass the outcome to Apple Pay. If the integration is expecting to redirect somewhere from `onSuccess` callback, then using this option would be advised.

Option #2. `onSuccess` callback implementation can just return Boolean value which would indicate the transaction outcome. Please note that with this option Apple Pay will get notified of the outcome only after the callback execution is finished. Thus, this option is not advised if the callback plans to trigger any redirects (unless the redirect is delayed).

For option #2 the wrapper allows `onSuccess` to be implemented in multiple ways depending on the code requirements/preferences:

- the `onSuccess` callback can be a normal, synchronous function which returns a Boolean
- or it can be an async function that returns a Boolean value
- or it can be a function that returns a `Promise` that gets resolved with a Boolean result

Once the Apple Pay controller has been create, the merchant's code needs to check whether Apple Pay is available. To do that they need to call `isAvailable` method of the controller instance. This method accepts a single optional argument `timeoutMs` - a timeout in milliseconds for the check. The method returns a `Promise` , which gets resolved if the Apple Pay is available and can be used, and gets rejected if it's not available or if the check timed out.

Once the Apple Pay availability has been confirmed the merchant's code has to call `attach` method of the controller instance to actually construct the Apple Pay button. The `attach` method takes no arguments and doesn't return anything.

Here is a quick example:

```
var applePayController = WindcavePayments.ApplePay.create({ ... },
function() { ... }, function(stage) { ... }, function() { ... });
applePayController.isAvailable(5000)
  .then(function() {
     // The Apple Pay is available
     applePayController.attach();
  })
  .catch(function() {
    // The Apple Pay is currently not available
  });
```

`options` object

The `options` object can contain a variety of different fields. Some of them are required, some of them are optional.

Required fields:

- `container` - the ID of the DOM element that will hold the Apple Pay button
- `url` - the URL that the Apple Pay token is going to be submitted to. This is the `ajaxSubmitApplePay` link from the Create Session response
- `mid` - the merchant identifier to use for the Apple Pay transaction
- `displayName` - the merchant's name

- `countryCode` - the merchant's two-letter ISO-3166 country code. Default value is NZ
- `currencyCode` - the transaction's three-letter ISO 4217 currency code
- `total` - the subobject containing the transaction totals
- `supportedNetworks` - the array of allowed card networks. Possible values are:
    - `amex`
    - `discover`
    - `interac`
    - `jcb`
    - `masterCard`
    - `visa`
- `requiredContactDetails` - the array of the type of address we want Apple Pay to ask from the user and pass to the AJAX Post URL. Possible values are:
    - `billing`
    - `shipping`

Optional fields:

- `button` - the subobject that contains settings for the button appearance. Details below.
- `env` - the environment the Apple Pay JS wrapper should work with. Possible values are `qa` , `dev` , `uat` , `sec` . Default value is `sec` . This environment is currently only used when 3DSecure is involved.
- `lineItems` - A set of line items that explain recurring payments and additional charges and discounts. This is an array of `LineItem` objects. Details below.

While the "`url`" is a required field in the options object, it does not have to be the final URL that the data is going to be submitted to. If the integration requires creating the session and thus obtaining `ajaxSubmitApplePay` link at a later stage, the initial value of "`url`" can be set to "https;//sec.windcave.com/mh". Then from the `onValidateMerchant` callback the integration can do async call to their back-end to create the session, get the `ajaxSubmitApplePay` link and update it in the `options` object.

`options.total` subobject

Available fields in this subobject:

- `amount` - the transaction amount
- `label` - a business name. Use the same business name people see when they look for the charge on their bank or credit card statement, for example, "`COMPANY, INC.`".

`options.button` subobject

Available fields in this subobject:

- `type` - the type of button to display. Default value is `buy` . More details here: https://developer.apple.com/documentation/apple_pay_on_the_web/applepaybuttontype

    -
    - `add-money` – An Apple Pay button useful for adding money to a card, account, or payment system.
    - `book` - An Apple Pay button useful for booking trips, flights, or other experiences.
    - `buy` - An Apple Pay button useful for product purchases.
    - `check-out` – An Apple Pay button useful for purchase experiences that include other payment buttons that start with "Check out."
    - `continue` - An Apple Pay button useful for general purchases.
    - `contribute` - An Apple Pay button useful to help people contribute money to projects, causes, organizations, and other entities.
    - `donate` - An Apple Pay button used by approved nonprofit organization that lets people make donations.
    - `order` - An Apple Pay button useful for placing orders for items such as meals or flowers.
    - `pay` - An Apple Pay button useful for paying bills or invoices.
    - `plain` - An Apple Pay button with the Apple Pay logo only, useful when an additional call to action isn't needed.
    - `reload` - An Apple Pay button useful for adding money to a card, account, or payment system.
    - `rent` - An Apple Pay button useful for renting items such as cars or scooters.
    - `set-up` – An Apple Pay button useful for prompting the user to set up a card.
    - `subscribe` - An Apple Pay button useful for purchasing a subscription such as a gym membership or meal-kit delivery service.
    - `support` - An Apple Pay button useful for helping people give money to projects, causes, organizations, and other entities.
    - `tip` - An Apple Pay button useful for letting people tip for goods or services.
    - `top-up` – An Apple Pay button useful for adding money to a card, account, or payment system.

- `style` - the appearance for the button. Default value is `black` . More details here: https://developer.apple.com/documentation/apple_pay_on_the_web/applepaybuttonstyle

    -
    - `black` - A black button with white lettering.
    - `white` - A white button with black lettering.
    - `white-outline` – A white button with black lettering and a black outline.

- `locale` - The desired button locale. The button locale type uses ISO-639-1 code for the language, followed by a hyphen (-), and the ISO-3166-1 code for the region. Default value is `en-US` . More details here: https://developer.apple.com/documentation/apple_pay_on_the_web/applepaybutton/3813986-locale

`options.lineItems` subobject

Available fields in this subobject:

- `label` - A required value that's a short, localized description of the line item.
- `amount` - A required value that's the monetary amount of the line item.
- `type` - A value that indicates whether the line item is final or pending.
- `paymentTiming` - The time that the payment occurs as part of a successful transaction.
- `recurringPaymentStartDate` - The date of the first payment.
- `recurringPaymentEndDate` - The date of the final payment.
- `recurringPaymentIntervalUnit` - The amount of time — in calendar units, such as day, month, or year — that represents a fraction of the total payment interval.
- `recurringPaymentIntervalCount` - The number of interval units that make up the total payment interval.
- `deferredPaymentDate` - The date, in the future, of the payment.
- `automaticReloadPaymentThresholdAmount` - The balance an account reaches before the merchant applies the automatic reload amount.

Full details on all those fields can be found on the official Apple Pay page: https://developer.apple.com/documentation/apple_pay_on_the_web/applepaylineitem

Examples

Here is a full example of how Apple Pay JS wrapper can be used:

```
<!DOCTYPE html>
<html>
  <head>
    <meta charset="utf-8">
    <meta http-equiv="X-UA-Compatible" content="IE=edge">
    <meta name="viewport" content="width=device-width,initial-scale=1.0">

    <script async src="https://sec.windcave.com/js/windcavepayments-
applepay-v1.js"></script>
  </head>
  <body>
    <div id="apple-pay-container"></div>

    <script>
      window.addEventListener('load', function() {
        var applePayOptions = {
          url:
"https://dev.windcave.com/mh/F6466B06A8A3BD7C4C11F3C5C2342253DA0B8E60934D46
C5C2186127737F71689CD090CF0557E268152E5C29FE47B6939",
          container: "apple-pay-container",
          displayName: "Test Store",
          countryCode: "NZ",
          currencyCode: "NZD",
          supportedNetworks: [ "visa", "mastercard", "amex" ],
          button: {
```

```
            type: "pay",
            style: "white-outline",
            locale: "en-US"
          },
          mid: "mid_19d26acb86bb4b34b3bead9eec1442e7"
          total: {
            amount: "38.06",
            label: "Test Store"
          },
          "lineItems": [
            {
                "label": "Bag Subtotal",
                "type": "final",
                "amount": "35.00"
            },
            {
                "label": "Free Shipping",
                "amount": "0.00",
                "type": "final"
            },
            {
                "label": "Estimated Tax",
                "amount": "3.06",
                "type": "final"
            }
          ]
        };

        window.applePayController =
WindcavePayments.ApplePay.create(applePayOptions, function(state, finalUrl,
outcomeNotificationFunction) {
          if (state == "done") {
            // Apple Pay transaction has been finished. Need to query the
session outcome now.
            ...
            return true; // in case of successful transaction, or return
false in case of failed one.
          }
        }, function(stage, errorMessage) {
          // Apple Pay has failed.
        });

        window.applePayController.isAvailable(5000)
          .then(function() {
            // Create and display the Apple Pay button
            window.applePayController.attach();
          }
          .catch(function() {
            // Apple Pay is currently not available
          });
      });
    </script>
  </body>
</html>
```